학습 노트: subprocess + asyncio (비동기 CLI 실행)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/`, `dev2/tools/common/`, `dev2/nodes/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 3 — CLI 도구 + 다국어 라우터 구현

한 줄 요약

Python 코드 안에서 pylint·bandit 같은 외부 검사 도구를 "동시에" 여러 개 돌리고, 모두 끝나면 결과를 한꺼번에 받아오는 방법.

개념 1. subprocess — 외부 CLI 도구 실행

(1) 비유

주방장(내 Python 코드)이 직접 모든 요리를 하지 않고, 전문 요리사(pylint, bandit)에게 "이 재료(파일)로 검사해줘"라고 시킨 뒤 완성된 접시(결과)만 받아오는 것.

요리사가 어떻게 일하는지는 몰라도 됩니다. 주문서(명령어)와 완성품(stdout)만 주고받으면 됩니다.

(2) 명확한 개념

> `subprocess`는 Python 프로그램이 다른 프로그램(외부 실행 파일)을 별도의 프로세스로 띄워서 실행하고, 그 출력(stdout/stderr)과 종료 코드(returncode)를 받아오는 표준 모듈이다.

- **stdout**: 프로그램이 정상적으로 내보낸 결과 (보통 JSON)
- **stderr**: 에러 메시지가 나오는 통로
- **returncode**: 0이면 정상 종료, 0이 아니면 이슈 발견 또는 오류

핵심 원리: 내 프로그램과 검사 도구는 완전히 분리된 프로세스다. 도구가 멈추거나 죽어도 내 프로그램은 살아있다. 그래서 안정적이다.

(3) 왜 쓰나

- 없으면: pylint, bandit, semgrep을 각각 Python 라이브러리로 직접 import해서 호출해야 함 → 도구마다 API가 다르고, 버전 올라가면 import 구조가 바뀌어서 코드가 깨짐
- 있으면: 터미널에서 치던 명령어(`pylint src/auth.py`) 그대로 코드에서 실행 → 도구 교체·추가가 쉽고, CLI 옵션을 그대로 재사용

(4) 실제 코드 (동기 버전 — 기본 이해용)

```
# dev2/tools/python/pylint_tool.py
import subprocess, json

def run_pylint_sync(files: list[str]) -> list[dict]:
    result = subprocess.run(
        ["pylint", *files, "--output-format=json"],  # 명령어를 "리스트"로 전달
        capture_output=True,   # 화면 출력 대신 result.stdout 으로 받기
        text=True,             # bytes 가 아니라 str 로 받기
        timeout=30,            # 30초 넘으면 TimeoutExpired 예외 (CLAUDE.md §6 타임
아웃 규칙)
    )
    if not result.stdout.strip():   # 이슈가 없으면 stdout 이 빔
        return []
    return json.loads(result.stdout)
```

보안 주의: `shell=True`는 절대 쓰지 않는다.

```
subprocess.run("pylint " + user_input, shell=True)   # 위험 — Shell Injection
subprocess.run(["pylint", user_input])               # 안전 — 인자가 명령어로 해
석되지 않음
```

개념 2. asyncio.create_subprocess_exec — 비동기로 실행하기

(1) 비유

동기 `subprocess.run`은 "요리사 한 명에게 시키고 그 앞에 서서 완성될 때까지 멍하니 기다리는 것". 요리가 5개면 한 명씩 차례로 시켜서 (5분 × 5 = 25분) 걸립니다.

`asyncio.create_subprocess_exec`는 "요리사 5명에게 동시에 주문하고, 나는 그동안 다른 일을 하다가 다 되면 한꺼번에 받는 것". 그래서 가장 오래 걸리는 요리 시간(5분)만 기다리면 됩니다.

(2) 명확한 개념

`asyncio.create_subprocess_exec`는 외부 프로그램을 비동기(코루틴) 로 실행한다. 도구가 일하는 동안 CPU는 멈춰서 기다리지 않고 다른 도구를 동시에 시작할 수 있다. `await proc.communicate()`로 출력이 다 나올 때까지 기다린 뒤 stdout/stderr를 받는다.

`subprocess.run` (동기) vs `create_subprocess_exec` (비동기) 차이:

구분	동기 subprocess.run	비동기 create_subprocess_exec
호출 방식	그냥 호출	`await` 필요
여러 개 실행	하나 끝나야 다음	`asyncio.gather`로 동시
5개 도구 (각 30초)	최대 150초	약 30초
결과 받기	`result.stdout`	`await proc.communicate()`

(3) 왜 쓰나

- 없으면: pylint(30초) → radon(20초) → bandit(30초) → semgrep(40초)를 순서대로 = 120초. CLAUDE.md §6의 "에이전트 30초 타임아웃" 안에 못 끝남
- 있으면: 4개를 동시에 시작 → 가장 느린 semgrep(40초)만 기다림 → 1개 에이전트 타임아웃(30초) 안에서 여러 도구를 굴릴 수 있음

왜 외부 CLI는 비동기 효과가 클까? 정적 분석 도구는 대부분 "내가 일하는 게 아니라 외부 프로그램이 디스크 읽고 분석하는 동안 기다리는" I/O 대기 시간이다. 이 대기 시간 동안 다른 도구를 동시에 굴리면 전체 시간이 크게 줄어든다.

(4) 실제 코드 (비동기 버전)

```
# dev2/tools/common/runner.py — 모든 언어 도구가 공통으로 쓰는 실행기
import asyncio, json

async def run_cli(cmd: list[str], timeout: int = 30) -> tuple[str, str, int]:
    """
    외부 CLI 도구를 비동기로 실행하고 (stdout, stderr, returncode)를 반환.
    타임아웃 발생 시 프로세스를 강제 종료하고 예외를 전파한다.
    """
    proc = await asyncio.create_subprocess_exec(
        *cmd,                                  # ["pylint", "auth.py", "--output-
format=json"]
        stdout=asyncio.subprocess.PIPE,     # 출력을 파이프로 받음
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, stderr = await asyncio.wait_for(proc.communicate(),
timeout=timeout)
    except asyncio.TimeoutError:
        proc.kill()                            # 안 죽이면 좀비 프로세스로 남음
        await proc.wait()
        raise                                  # CLAUDE.md §6: with 블록 바깥에서 잡도
록 전파
    return stdout.decode(), stderr.decode(), proc.returncode
```

```python
# 여러 도구를 동시에 실행 (개념 통합)
async def run_python_tools(files: list[str]) -> list[dict]:
    results = await asyncio.gather(
        run_cli(["pylint", *files, "--output-format=json"]),
        run_cli(["bandit", "-r", *files, "-f", "json"]),
        run_cli(["radon", "cc", *files, "-j"]),
        return_exceptions=True,   # 하나 실패해도 나머지 결과는 살림
    )
    issues: list[dict] = []
    for r in results:
        if isinstance(r, Exception):
            continue              # 타임아웃/실패한 도구는 건너뜀
        stdout, _, _ = r
        if stdout.strip():
            issues.extend(json.loads(stdout))
    return issues
```

3rd-Project-sub2 프로젝트에서

파일: `dev2/tools/common/runner.py` (공통 실행기), `dev2/tools/python/· javascript/ ·csharp/` (언어별 도구 래퍼)

역할: 1. 다국어 라우터가 "이 PR에 .py 파일이 3개, .js 파일이 1개 있다"고 알려주면 2. 각 언어 폴더의 도구 래퍼가 `run_cli()`를 호출해 pylint/eslint 등을 비동기 동시 실행 3. 모든 결과를 모아 Output Contract A (`AgentOutput`, `schema.py`)로 변환 → Dev4에게 전달

> CLAUDE.md §6 타임아웃 규칙 연결: `try/except asyncio.TimeoutError`는 `async with AgentHarness()` 바깥에 둬야 `agent_traces.status`가 잘못 'success'로 기록되지 않는다.

연관 개념

- **asyncio.gather** → 여러 코루틴을 동시 실행하고 결과를 리스트로 모음 (`docs/learning/python/asyncio_gather.md`)
- **asyncio.wait_for** → 코루틴에 타임아웃을 씌우는 함수. 시간 초과 시 `TimeoutError`
- 좀비 프로세스 → 부모가 거두지 않은 끝난 자식 프로세스. `proc.kill()` 후 `await proc.wait()`로 정리
- **PIPE** → 프로세스 간 데이터가 흐르는 관. stdout을 PIPE로 받아야 코드에서 읽을 수 있음

복습 퀴즈 (3문항)

1. (이해) `subprocess.run`(동기)과 `asyncio.create_subprocess_exec`(비동기)의 가장 큰 차이는 무엇이며, 정적 분석 도구 5개를 돌릴 때 왜 비동기가 유리한가?

- 답: 동기는 한 도구가 끝나야 다음을 시작(순차)하고, 비동기는 `asyncio.gather`로 동시 실행 가능. 정적 분석은 외부 프로그램이 일하는 I/O 대기 시간이 길어서, 동시에 굴리면 가장 느린 도구 시간만큼만 걸려 30초 타임아웃 안에 들어온다.

2. **(적용)** `await asyncio.wait_for(proc.communicate(), timeout=30)`에서 30초가 지나면 무슨 일이 일어나며, 그다음 코드에서 반드시 해야 할 두 가지는?
   - 답: `asyncio.TimeoutError`가 발생한다. (1) `proc.kill()`로 멈춘 도구 프로세스를 강제 종료하고 (2) `await proc.wait()`로 좀비 프로세스를 정리한 뒤 예외를 전파한다.
3. **(비교/보안)** `subprocess.run("pylint " + user_input, shell=True)`와 `run_cli(["pylint", user_input])` 중 안전한 것은? 그 이유는?
   - 답: 리스트로 전달하는 후자가 안전. `shell=True`는 사용자 입력이 셸 명령으로 해석되어 `; rm -rf /` 같은 악성 명령이 끼어드는 Shell Injection이 가능하지만, 리스트 인자는 pylint의 단순 인자로만 전달되어 명령으로 실행되지 않는다.